Mês 10 — Biblioteca Consolidada + Repositório + Redação (jun/2027)

Bolsista PIBITI

Mês 10

Sumário

1 O que o plano exige **2**

2 Estrutura da biblioteca **3**

2.1 Estrutura de diretórios 3
2.2 O arquivo `__init__.py` 3
2.3 O módulo `wpt.py` 3
2.4 O módulo `descritores.py` 4
2.5 O módulo `extracao.py` 4
2.6 O módulo `metricas.py` 5
2.7 O módulo `utils.py` 5

3 Modernizando com `pyproject.toml` **6**

4 API Simples **7**

5 Testes Unitários **8**

6 Documentação **10**

7 Organizar Repositório **11**

8 Ambiente Python (`environment.yml` e `requirements.txt`) **12**

9 Rascunho do Artigo **13**

10 Gerar Tabelas LaTeX Automaticamente **14**

11 Checklist **15**

12 Desafio Final **16**

Capítulo 1

O que o plano exige

Neste décimo mês, nosso principal objetivo é consolidar todo o código desenvolvido em uma biblioteca organizada e documentada e organizar o repositório do projeto. O plano também prevê o início da redação do artigo para submissão.

Objetivo: Consolidar o produto tecnológico central do PIBITI e começar a redação científica.

Atividades principais:

- Consolidar a biblioteca de extração de características (WPT + descritores de contorno) com documentação.
- Estruturar o repositório final com código, arquivos de partição, `README.md`, ambientes virtuais e resultados experimentais.
- Garantir a não distribuição dos arquivos de áudio diretamente (por direitos e tamanho).
- Elaborar um rascunho de artigo para conferências como ENIAC, CONIC/SEMESP ou REIC.

Marco / Entrega

Entrega do Mês: Biblioteca Python documentada + repositório bem estruturado + rascunho do artigo.

Capítulo 2

Estrutura da biblioteca

Uma biblioteca Python moderna precisa ser modular. Vamos criar o pacote `wavelet_features`.

2.1 Estrutura de diretórios

Estrutura do pacote

```
wavelet_features/
          __init__.py
          wpt.py
          descritores.py
          extracao.py
          metricas.py
          utils.py
```

2.2 O arquivo `__init__.py`

wavelet_features/__init__.py

```
"""
Wavelet Features - Biblioteca para extra  o de caracter sticas
   em  udios  usando WPT.
"""
from .extracao import extrair_features

__version__ = "0.1.0"
__all__ = ["extrair_features"]
```

2.3 O módulo `wpt.py`

wavelet_features/wpt.py

```
import pywt
import numpy as np

```

```python
def aplicar_wpt(audio, wavelet='db4', maxlevel=4):
    """
    Aplica a Transformada Wavelet em Pacotes (WPT).
    """
    wp = pywt.WaveletPacket(data=audio, wavelet=wavelet, mode='
        symmetric', maxlevel=maxlevel)
    return wp

def extrair_coeficientes_folha(wp):
    """
    Extrai coeficientes dos n s folha.
    """
    nodes = wp.get_level(wp.maxlevel, order="freq")
    return [n.data for n in nodes]
```

2.4 O módulo descritores.py

wavelet_features/descritores.py

```python
import numpy as np
from scipy.stats import skew, kurtosis

def calcular_descritores_estatisticos(coeficientes):
    """
    Calcula media, variancia, skewness e kurtosis.
    """
    descritores = {}
    for i, coef in enumerate(coeficientes):
        prefix = f"node_{i}_"
        descritores[prefix + "mean"] = np.mean(coef)
        descritores[prefix + "var"] = np.var(coef)
        descritores[prefix + "skew"] = skew(coef)
        descritores[prefix + "kurtosis"] = kurtosis(coef)
    return descritores
```

2.5 O módulo extracao.py

wavelet_features/extracao.py

```python
from .wpt import aplicar_wpt, extrair_coeficientes_folha
from .descritores import calcular_descritores_estatisticos

def extrair_features(audio, wavelet='db4', level=4):
    """
    Pipeline completo: aplica WPT e calcula descritores.
    """
    wp = aplicar_wpt(audio, wavelet, level)
    coefs = extrair_coeficientes_folha(wp)
    features = calcular_descritores_estatisticos(coefs)
```

```
    return features
```

2.6 O módulo `metricas.py`

wavelet_features/metricas.py

```
from sklearn.metrics import accuracy_score, precision_score,
   recall_score, f1_score

def calcular_metricas(y_true, y_pred):
    """
    Calcula as principais m tricas de classifica  o.
    """
    return {
        "accuracy": accuracy_score(y_true, y_pred),
        "precision": precision_score(y_true, y_pred),
        "recall": recall_score(y_true, y_pred),
        "f1": f1_score(y_true, y_pred)
    }
```

2.7 O módulo `utils.py`

wavelet_features/utils.py

```
import librosa

def carregar_audio(caminho, sr=16000):
    """
    Carrega o arquivo de  udio  usando librosa.
    """
    y, _ = librosa.load(caminho, sr=sr)
    return y
```

Capítulo 3

Modernizando com `pyproject.toml`

pyproject.toml

```toml
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "wavelet_features"
version = "0.1.0"
description = "Biblioteca para detec  o de deepfakes de  udio usando WPT."
authors = [{ name="Thiago Schiavi" }]
dependencies = [
    "numpy>=1.21.0",
    "scipy>=1.7.0",
    "PyWavelets>=1.3.0",
    "librosa>=0.9.0",
    "scikit-learn>=1.0"
]

[project.optional-dependencies]
dev = ["pytest", "black", "flake8"]
```

Dica

Para instalar localmente em modo de edição, use `pip install -e .` na raiz do projeto.

Capítulo 4

API Simples

exemplo_uso.py

```python
from wavelet_features import extrair_features
from wavelet_features.utils import carregar_audio

audio_signal = carregar_audio("dataset/real/amostra.wav")
features = extrair_features(audio_signal, wavelet='db4', level=4)

print(f"Total de features extra das: {len(features)}")
for k, v in list(features.items())[:5]:
    print(f"{k}: {v}")
```

Capítulo 5

Testes Unitários

tests/test_wpt.py

```python
import numpy as np
from wavelet_features.wpt import aplicar_wpt,
    extrair_coeficientes_folha

def test_aplicar_wpt():
    audio = np.random.randn(16000)
    wp = aplicar_wpt(audio, maxlevel=2)
    assert wp.maxlevel == 2

def test_extrair_coeficientes_folha():
    audio = np.random.randn(16000)
    wp = aplicar_wpt(audio, maxlevel=2)
    coefs = extrair_coeficientes_folha(wp)
    assert len(coefs) == 4
```

tests/test_descritores.py

```python
import numpy as np
from wavelet_features.descritores import
    calcular_descritores_estatisticos

def test_calcular_descritores():
    coefs = [np.random.randn(100), np.random.randn(100)]
    descritores = calcular_descritores_estatisticos(coefs)
    assert 'node_0_mean' in descritores
    assert 'node_1_var' in descritores
```

tests/test_extracao.py

```python
import numpy as np
from wavelet_features.extracao import extrair_features

def test_extrair_features():
    audio = np.random.randn(16000)
    features = extrair_features(audio, level=2)
    assert isinstance(features, dict)
    assert len(features) == 16 # 4 nos * 4 descritores
```

Rodando testes

```
pytest tests/
```

Capítulo 6

Documentação

README.md

```
# Wavelet Features for Audio Deepfake Detection

Biblioteca desenvolvida para extra  o de caracter sticas usando
   Wavelet Packet Transform (WPT) no contexto de detec  o de
    udios  deepfake.

## Instala  o

```bash
git clone <url_do_repositorio>
cd <repositorio>
pip install -e .
```

Uso

```python
from wavelet_features import extrair_features
import numpy as np

audio = np.random.randn(16000)
features = extrair_features(audio, level=4)
```

Contribui o
Bolsista: Thiago Schiavi
Projeto PIBITI.
```

Capítulo 7

Organizar Repositório

Estrutura final recomendada

```
meu_projeto_pibiti/
          .gitignore
          README.md
          pyproject.toml
          environment.yml
          requirements.txt
          data/
                particoes/ (CSVs de treino/teste)
                descritores/ (CSVs gerados)
          notebooks/ (EDA, analises)
          src/ (Scripts de treino)
          tests/
          wavelet_features/ (Biblioteca)
```

.gitignore

```
# Python
__pycache__/
*.py[cod]
*.so
.env
.venv
env/
venv/
*.egg-info/

# Dados (NAO comitar audios)
data/audio/
*.wav
*.flac
*.mp3

# Jupyter
.ipynb_checkpoints
```

Capítulo 8

Ambiente Python (`environment.yml` e `requirements.txt`)

environment.yml

```yaml
name: deepfake_env
channels:
  - conda-forge
  - defaults
dependencies:
  - python=3.9
  - numpy
  - scipy
  - scikit-learn
  - matplotlib
  - seaborn
  - pywavelets
  - jupyterlab
  - pytest
  - pip
  - pip:
    - librosa
    - xgboost
    - lightgbm
```

requirements.txt

```
numpy>=1.21.0
scipy>=1.7.0
scikit-learn>=1.0
matplotlib>=3.4
seaborn>=0.11
PyWavelets>=1.3.0
librosa>=0.9.0
xgboost>=1.5.0
lightgbm>=3.3.0
pytest>=7.0.0
```

Capítulo 9

Rascunho do Artigo

Estrutura Base para ENIAC / REIC

Título: Detecção de Deepfakes de Áudio utilizando Wavelet Packet Transform e Descritores de Contorno
Resumo: Abordar contexto, problema, proposta e melhores resultados.
1. Introdução: O que são deepfakes de áudio, impactos, e o uso de WPT.
2. Trabalhos Relacionados: Outras técnicas como MFCC, LFCC, e modelos profundos.
3. Metodologia:

- Dataset ASVspoof
- WPT e extração
- Classificadores tradicionais

4. Experimentos e Resultados: Tabelas de métricas, análise do nível da WPT e impacto da família Wavelet.
5. Conclusão: Resumo dos achados e trabalhos futuros.

Capítulo 10

Gerar Tabelas LaTeX Automaticamente

gerar_tabelas.py

```python
import pandas as pd

def csv_para_latex(csv_path, output_path):
    df = pd.read_csv(csv_path)
    # Arredondando
    df = df.round(4)
    latex_str = df.to_latex(index=False, caption="Resultados da Classifica  o", label="tab:resultados")

    with open(output_path, "w") as f:
        f.write(latex_str)
    print(f"Tabela LaTeX salva em {output_path}")

# Exemplo:
# csv_para_latex("data/descritores/resultados_modelos.csv", "tabela_resultados.tex")
```

Saída esperada

Um arquivo `.tex` com a formatação do Pandas usando `\begin{tabular}` que pode ser incluído no artigo via `\input{tabela_resultados.tex}`.

Capítulo 11

Checklist

Todos os módulos de `wavelet_features` implementados e comentados.

`pyproject.toml` configurado.

Testes unitários escritos e passando.

Repositório limpo, sem arquivos de áudio.

`.gitignore` correto.

`environment.yml` atualizado.

Rascunho inicial do artigo esboçado.

Capítulo 12

Desafio Final

Desafio

Adicione cobertura de código (*coverage*) aos seus testes unitários. Execute:

```
pip install pytest-cov
pytest --cov=wavelet_features tests/
```

Tente alcançar 90% de cobertura!